

Técnicas Digitales 2 - Trabajo práctico N° 2 - Act N°5

Braida Agustín (404621) - Ernst Pedro (400624) - Polizze Tomas (400793) - Valentino Rao (402308)

**Técnicas Digitales 2 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional**

I. RESUMEN

En esta actividad se implementa el control de potencia de salida mediante Modulación por Ancho de Pulso (PWM) por hardware empleando los temporizadores de propósito general del microcontrolador STM32F103 (Cortex-M3). Se utiliza el periférico TIM2 canal 3 asociado al pin PA2 para comandar la intensidad luminosa de un diodo LED externo. El periférico genera la señal modulada de forma autónoma en hardware mediante la configuración de los registros de pre-escalador (PSC), autorrecarga (ARR) y modo de comparación de salida (CCMR2). Sobre esta base, se programa un efecto de degradado u orgánico ("fading") controlado por software mediante la actualización periódica del registro de captura/comparación (CCR3).

II. INTRODUCCIÓN

La Modulación por Ancho de Pulso (PWM) es una técnica altamente eficiente para regular la energía entregada a una carga eléctrica sin incurrir en pérdidas por disipación lineal. En microcontroladores de arquitectura ARM Cortex-M3 como el STM32F103, los temporizadores de propósito general (TIMx) disponen de canales de comparación de salida que generan señales PWM puras directamente por hardware, liberando al procesador principal del trabajo de conmutación de pines.

El presente informe detalla la configuración a nivel de registros CMSIS del periférico TIM2 operando a una frecuencia de reloj de 72 MHz, el cálculo de los valores de temporización y la implementación del algoritmo de variación dinámica del ciclo de trabajo (*Duty Cycle*) para lograr el efecto de "fading".

III. CONFIGURACIÓN DEL PERIFÉRICO HARDWARE TIM2 (PWM)

Para mapear la salida del canal 3 del temporizador TIM2 al pin PA2, se requiere habilitar el reloj del periférico y configurar la función alternativa en el registro del puerto GPIO correspondiente.

III-A. Configuración del Pin GPIO (PA2)

El pin PA2 debe configurarse en modo de salida de función alternativa Push-Pull a 50 MHz:

- Bits $MODE2[1:0] = 11$ (Salida a 50 MHz)
- Bits $CNF2[1:0] = 10$ (Alternate Function Output Push-Pull)

Esto se logra aplicando la máscara $0 \times B$ a la posición del pin PA2 dentro del registro $GPIOA \rightarrow CRL$.

III-B. Cálculo del Pre-escalador (PSC) y Autorrecarga (ARR)

Con la arquitectura del sistema funcionando a 72 MHz, el reloj del bus APB1 es de 36 MHz. Debido al multiplicador interno de reloj del temporizador de la arquitectura STM32 (cuando el prescalador $APB1 > 1$), la frecuencia de entrada al contador de TIM2 es de $f_{TIM} = 72$ MHz.

Se establece una frecuencia de señal PWM de 1 kHz ($T_{PWM} = 1$ ms):

1. **Pre-escalador (TIM2→PSC):** Se configura en 71 decimal, lo que divide el reloj por 72:

$$f_{CNT} = \frac{72 \text{ MHz}}{71 + 1} = 1 \text{ MHz} \Rightarrow T_{CNT} = 1 \mu s$$

2. **Registro de Autorrecarga (TIM2→ARR):** Se fija en 999 decimal, logrando un período total de 1000 tics:

$$T_{PWM} = (999 + 1) \times 1 \mu s = 1000 \mu s = 1 \text{ ms} \quad (1 \text{ kHz})$$

III-C. Configuración del Modo PWM en CCMR2 y CCR

Para habilitar el canal 3 en modo PWM 1 (activo mientras el contador sea menor a CCR3):

- **Registro TIM2→CCMR2:** Se establecen los bits $OC3M[2:0] = 110$ (PWM Mode 1) y el bit $OC3PE = 1$ (Preload enable en el registro de comparación).
- **Registro TIM2→CCER:** Se activa la salida del canal 3 ajustando $CC3E = 1$.
- **Control del Temporizador TIM2→CR1:** Se activa el bit $ARPE = 1$ (Auto-reload preload enable) y se enciende el contador con $CEN = 1$.

IV. EFECTO FADING POR SOFTWARE

Una vez sintetizada la señal PWM por hardware en el pin PA2, el control de intensidad luminosa se efectúa modificando dinámicamente el contenido del registro de comparación TIM2→CCR3.

- Un valor de CCR3 = 0 representa un *Duty Cycle* del 0% (LED apagado).
- Un valor de CCR3 = 999 representa un *Duty Cycle* del 100% (LED a máxima intensidad).

Mediante una base de tiempo generada por el temporizador SysTick a 1 ms, se incrementa o decrementa progresivamente el valor de CCR3 en pasos discretos, invirtiendo la dirección al alcanzar los límites extremos del contador (0 y 999).

V. CÓDIGO FUENTE

A continuación se adjunta el código fuente en C desarrollado a nivel de registros CMSIS que implementa la solución.

```
1 #include "stm32f1xx.h"
2 #include <stdint.h>
3
4 volatile int16_t pwm_duty = 0;
5 volatile int8_t pwm_step = 5;
6 volatile uint32_t tick_fade = 0;
7
8 void SysTick_Handler(void) {
9     // Interrupcion cada 1 ms (1 kHz)
10    tick_fade++;
11    if (tick_fade >= 10) { // Actualizar
12        fading cada 10 ms
13        tick_fade = 0;
14
15        pwm_duty += pwm_step;
16        if (pwm_duty >= 999) {
17            pwm_duty = 999;
18            pwm_step = -pwm_step;
19        } else if (pwm_duty <= 0) {
20            pwm_duty = 0;
21            pwm_step = -pwm_step;
22        }
23
24        // Variacion del Duty Cycle por
25        software escribiendo en CCR3
26        TIM2->CCR3 = (uint32_t)pwm_duty;
27    }
28
29 int main(void) {
30     // 1. Habilitar Clocks (GPIOA, TIM2, AFIO)
31     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
32     RCC_APB2ENR_AFIOEN;
33     RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
34
35     // 2. Configurar PA2 como Alternate
36     Function Push-Pull (50MHz)
37     GPIOA->CRL &= ~(0xF << 8);
38     GPIOA->CRL |= (0xB << 8);
39
40     // 3. Configurar TIM2 (Prescaler = 71 -> 1
41     MHz, ARR = 999 -> 1kHz PWM)
42     TIM2->PSC = 71;
43     TIM2->ARR = 999;
44
45     // 4. Configurar Modo PWM 1 en Canal 3 (
46     CCMR2) y habilitar salida (CCER)
47     TIM2->CCMR2 &= ~(TIM_CCMR2_OC3M);
48     TIM2->CCMR2 |= (6 << TIM_CCMR2_OC3M_Pos) |
49     TIM_CCMR2_OC3PE;
50     TIM2->CCER |= TIM_CCER_CC3E;
51
52     // 5. Habilitar registros de Preload y
53     encender TIM2
```

```
47 TIM2->CR1 |= TIM_CR1_ARPE | TIM_CR1_CEN;
48 TIM2->EGR |= TIM_EGR_UG;
49
50 // 6. Configurar SysTick a 1 ms (72 MHz)
51 SysTick->CTRL |=
52 SysTick_CTRL_CLKSOURCE_Msk;
53 SysTick->LOAD = 71999;
54 SysTick->VAL = 0;
55 SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk
56 | SysTick_CTRL_ENABLE_Msk;
57
58 while (1) {
59     // El hardware genera el PWM
60     autonomamente en PA2.
61     // El software solo actualiza TIM2->
62     CCR3 via SysTick.
63 }
```

VI. PRUEBAS Y VERIFICACIÓN

Se verificó el funcionamiento del sistema en la placa de desarrollo comprobando que:

- La generación de la señal de modulación se realiza de forma totalmente transparente sin consumo de ciclos de CPU en el bucle principal.
- El ciclo de trabajo de la señal en el pin PA2 varía fluidamente entre el 0% y el 100%, logrando un efecto de gradiente de luz continuo.
- La frecuencia portadora de 1 kHz garantiza la total ausencia de parpadeo visual.

VII. CONCLUSIÓN

El uso de temporizadores por hardware para la generación de señales PWM constituye la práctica estándar en sistemas embebidos industriales. La combinación de periféricos autónomos (TIM2) para la síntesis de la onda y de interrupciones livianas (SysTick) para el control de alto nivel permite maximizar la eficiencia energética del microcontrolador y la previsibilidad temporal de la aplicación.